

Plan de implementación integral — Procurador SCW

Documento de planificación · 2026-07-11 · Estado: **en ejecución — Fases 1 y 2 completadas (2026-07-13)**

Regla de oro: **no romper el funcionamiento del sistema en producción con ninguna implementación.**

Fuentes: [docs/internal/informe-bugs-2026-07.md](#) , [docs/internal/informe-pendientes-2026-07-11.pdf](#) (ver también [informe-pendientes-2026-07-13.pdf](#) , versión actualizada), [docs/internal/plan-seguridad-precomercializacion-2026-07.md](#) , [docs/internal/flujo-staging-rollback.md](#) , [docs/internal/runbook-comandos.md](#) .

☒ Estado de ejecución (actualizado 2026-07-13)

Fase 1 — Infraestructura: COMPLETA. D5 (limpieza) ☒ · D1 (ya estaba resuelto, verificado) ☒ · D3 backend (`npm audit fix` : 19→1 vulnerabilidades; se detectó y corrigió en el camino un drift real de `multer` faltante en `package.json`) ☒ · D4 queda diferido a antes de B3 (por diseño del plan).

Fase 2 — Bugs + QA: COMPLETA.

- **Lote 2A** (cobranza: C1, C2, A1, M4, A4, A2, A3) ☒ deployado a prod. C1/C2/A1/M4 verificados **E2E con un pago sandbox real** — el E2E cazó un bug adicional (conflicto de tipos `timestamp` vs `timestamptz` en `applyTrialBonus`) que solo se manifestaba al aplicar un pago real.

- **Lote 2B** (cuotas: M1, M2/SEC-4, M3, B2, B4, B5, B8) ☒ deployado a prod. M2/SEC-4 y M3 verificados E2E en staging. **SEC-4 queda resuelto** (era el mismo bug que M2).

- **Lote 2C** (Electron: A5, A6, M5, M6, M10, B1, B6; M8 mitigado) ☒ **release v2.7.37 publicado** en GitHub Releases y confirmado en vivo — llega a los usuarios vía auto-updater.

- **Lote 2D** (U9.3) ☒ investigado y **resuelto: no era un bug de producto**, sino un artefacto de automatización (`confirm()` nativo + navegación externa a MP bloqueaban las herramientas de testing, no a un usuario real). Confirmado con test API en staging (200 + `init_point` en 373ms).

- **Extra:** mejora de UX del checkout (alert/confirm → toast/modal no bloqueante) implementada y deployada, cierra la recomendación secundaria del Hallazgo #2.

Próximo: Fase 3 — Seguridad (solo quedan SEC-1 auditoría y SEC-2 CI; SEC-4 ya cerrado).

0. Objetivo y alcance

Ordenar en fases ejecutables toda la deuda técnica y las mejoras pendientes, respetando dependencias reales y minimizando el riesgo sobre producción. El orden sigue el criterio pedido:

1. **Fase 1 — Infraestructura técnica** (4 pendientes: D1, D3, D4, D5)
2. **Fase 2 — Corrección de bugs relevados + flecos de QA integral**
3. **Fase 3 — Seguridad pre-comercialización** (SEC-1, SEC-2, SEC-4)
4. **Fase 4 — Cuentas / contratos externos** (B3, AZ, Facturante)
5. **Fase 5 — Diferidos al lanzamiento público** (L1, L2)
6. **Fase 6 — Propuestas de producto sin aprobar** (módulo Bitácora)

Dos acoplamientos que alteran ligeramente el orden lineal (explicados abajo): (a) el paquete de bugs de cobro **C2+A1+M4** es prerequisite duro de **B3** (Fase 4) → debe cerrarse en la Fase 2; (b) **SEC-4** (Fase 3) es el mismo problema que el bug **M2** → su parte de código se resuelve dentro del lote backend de la Fase 2, dejando para la Fase 3 solo la auditoría (SEC-1) y el CI/verificación diaria (SEC-2).

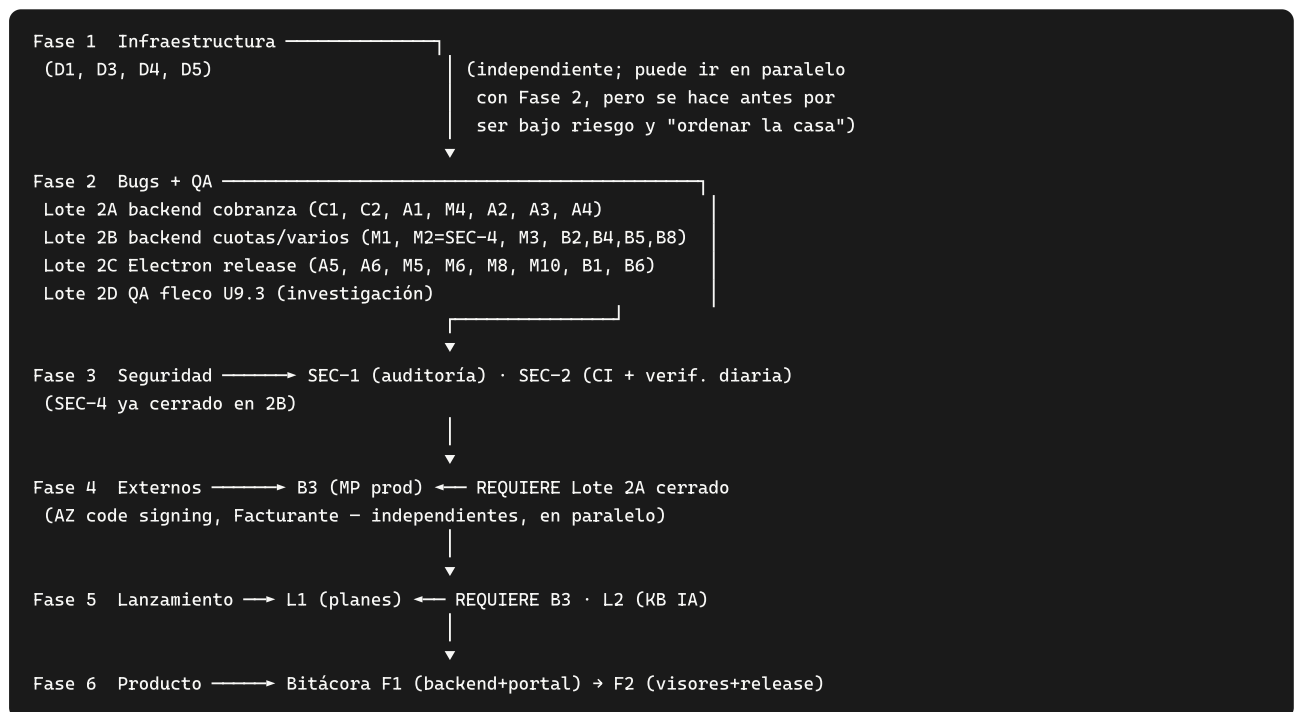
1. Principios de ejecución segura (aplican a TODAS las fases)

Estos principios son la implementación concreta de la regla de oro. Ningún cambio llega a producción sin pasar por ellos.

1. **Staging primero, siempre.** Todo cambio de backend se prueba en `staging-api.procuradortool.com` (DB `procurador_db_staging` , MP sandbox fijo) antes de tocar producción. Todo cambio de Electron se prueba con `npm start` y `npm run build:dir` antes de publicar release.

2. **Un cambio lógico por deploy.** No se agrupan correcciones no relacionadas en el mismo deploy. Si algo falla, se sabe exactamente qué lo causó y el rollback es quirúrgico.
3. **Backup pre-deploy obligatorio.** Antes de cada deploy a producción: `ops/backup-now.sh prod` (backup local on-demand de la DB). El backup rutinario `.7z` en automatización se hace al inicio y al cierre de cada sesión de trabajo.
4. **Migraciones de DB aditivas y reversibles.** Nada de `DROP COLUMN` / `DROP TABLE` en caliente. Cada migración con su nota de rollback. Las columnas nuevas nacen con default seguro.
5. **Smoke test después de cada deploy.** `smoke-test-pjn.js` (48 checks) + verificación del endpoint tocado. Si falla, rollback inmediato.
6. **Rollback de 3 capas definido y probado** (ya validado en simulacros): datos (`ops/restore-db.sh`), código backend (`git checkout &tag& && pm2 restart`), app Electron (fix-forward con versión nueva). Detalle en `docs/internal/flujo-staging-rollback.md`.
7. **Tag git antes de cada lote.** Un tag `pre-<lote>-<fecha>` antes de empezar cada agrupación, para poder volver al punto exacto.

2. Mapa de fases y dependencias



Regla de dependencia dura: **B3** (cobro real) **no se activa** hasta que el Lote 2A esté en producción y verificado. Cobrar dinero real con C2/A1/M4 sin corregir = pagos perdidos y suscripciones mal seteadas.

3. Detalle por fase

FASE 1 — Infraestructura técnica

Bajo riesgo, "ordena la casa" antes de tocar lógica. Se hace primero porque es barato y reduce ruido.

Paso	Ítem	Acción	Riesgo	Rollback
1.1	D5	Borrar temporales muertos (<code>seed_legal_tmp.js</code> , <code>test_legal_tmp.js</code> , <code>test_legal_full_tmp.js</code>) + clasificar untracked del repo (<code>ext-header-preview.png</code> , <code>tests/*.png</code> , <code>tests/tests/</code> , <code>imagenes/old/</code>)	Nulo (código muerto)	<code>git revert</code>
1.2	D1	Migración <code>ALTER DEFAULT PRIVILEGES ... GRANT ALL ON TABLES TO procurador_user</code> (ya se aplicó una versión en <code>20260627_grant ...</code> ; verificar cobertura y completar)	Bajo (solo permisos)	Migración reversible
1.3	D3	<code>npm audit fix</code> (sin <code>--force</code>) en <code>backend-server</code> y <code>electron-app</code> . Backend primero en staging → smoke → prod. Electron: fix + <code>npm start</code> + build:dir + smoke local → release	Medio (cambia deps transitorias)	Backend: restaurar <code>package-lock.json</code> + <code>npm ci</code> . Electron: fix-forward
1.4	D4	<code>npm audit fix --force</code> (breaking: mercadopago/uuid, axios, undici, electron). Solo en staging primero , con prueba E2E completa del flujo de pagos MP sandbox. Diferible hasta antes de B3	Alto (breaking changes en la lib de pagos)	Restaurar lock + <code>npm ci</code> ; probar todo el checkout otra vez

Orden interno recomendado: D5 → D1 → D3 → (D4 se puede diferir a justo antes de la Fase 4, para que la actualización de la lib de MercadoPago quede fresca al activar B3).

FASE 2 — Corrección de bugs + flecos de QA

Se agrupa por naturaleza de deploy para respetar "un cambio lógico por deploy" sin multiplicar releases de Electron.

Lote 2A — Backend, cobranza (crítico, prerequisite de B3)

Solo backend, deployable sin tocar la app. Cada uno es su propio deploy a staging → prod.

Orden	Bug	Fix resumido	Migración DB
1	C1	En el gate de <code>log-execution</code> , comparar <code>\${usageCol} < effectiveLimit</code> (no sumar bonus de nuevo a la izquierda). Una línea.	No
2	C2	Deduplicar webhooks por "procesado con éxito" (<code>processed_at IS NOT NULL</code>), no por existencia del id. Permitir reprocesar el <code>approved</code> que llega tras un <code>pending</code> .	No (usa <code>webhook_events.processed_at</code> existente)
3	A1	<code>applyTrialBonus</code> / <code>applyRenewal</code> (o el UPDATE del webhook) setean explícitamente <code>status='active'</code> , <code>next_billing_date</code> , <code>last_payment_at</code> .	No
4	M4	Envolver el flujo de pago aprobado (insert pago → apply → update sub → update user) en una transacción.	No
5	A4	<code>/auth/forgot-password</code> : escapar el email reflejado, respuesta genérica exista o no la cuenta, aplicar <code>loginLimiter</code> .	No
6	A2	Claim-por-ventana: agregar techo temporal + limpiar <code>checkout_initiated_at</code> al vincular.	No
7	A3	<code>linkPreapproval</code> : validar que el preapproval sea atribuible al usuario antes del UPDATE.	No

C2+A1+M4 se prueban juntos en staging con un ciclo de pago sandbox completo (pending→approved, cancelar en primer período, reactivar) porque su interacción es el punto sensible.

Lote 2B — Backend, cuotas y varios

Solo backend. SEC-4 se cierra aquí (es M2).

Bug	Fix resumido
M2 / SEC-4	Mover el enforcement del tope de trial (20 usos) al camino server-side que toda ejecución atraviesa (<code>/license/execution/start</code>) y/o la rama de submódulo de <code>log-execution</code>), espejando el gate.
M1	Condicionar el reset mensual al ciclo de facturación real y a <code>payment_provider</code> (no resetear trials activos sin pago ni duplicar el reset de pagos).
M3	Hacer atómica la adquisición del lock de ejecución (409 dentro del upsert o <code>SELECT ... FOR UPDATE</code>); corregir el mensaje "30 minutos" → 5 min.
B2	<code>PUT /usuarios/api/profile</code> : normalizar el guard para no crashear con <code>null</code> .
B4	<code>verify-session</code> : tratar <code>expires_at IS NULL</code> como no-expirado.
B5	<code>POST /monitor/log</code> : validar pertenencia de <code>parte_id</code> ; considerar tope server-side de novedades.
B8	Rate-limit del chat IA: podar el Map / persistir el contador.

Lote 2C — App Electron (una sola release)

Requiere bump de versión + tag + `npm run release` . Se agrupan todos los bugs de la app para no fragmentar releases.

Bug	Fix resumido
A5 + A6	Guard de concurrencia único por <code>authManager.activeChild</code> seteado antes de los awaits; en <code>before-quit</code> matar <code>activeChild</code> y llamar <code>shutdown()</code> .
M5	Limpiar <code>_lastKnownCuit</code> en <code>logout</code> .
M6	<code>list-expedientes</code> con <code>buildRunEnv(cuit)</code> + lock + headless (igual que los otros handlers).
M8	No persistir el JWT en texto plano en <code>config_monitoreo.json</code> (o limpiarlo garantizadamente al terminar).
M10	<code>isSigtermError()</code> reconoce "Código null" como detención voluntaria.
B1	<code>run-process-custom-date</code> : restaurar la config al arrancar si quedó un <code>.backup</code> de una corrida abortada.
B6	Validar esquema en <code>open-external-url</code> y no pasar <code>url</code> como flag en <code>open-url-in-chrome</code> .

Lote 2D — Fleco de QA bloqueante

Ítem	Acción
U9.3	Reproducir en staging con MP sandbox el flujo "pagar una reactivación" y encontrar la causa raíz (no es el bug ya corregido). Puede resultar en un fix backend adicional. Los flecos menores de QA (A1.14, A7.5, etc.) son confirmaciones visuales, no bugs → se cierran por verificación, sin código.

FASE 3 — Seguridad pre-comercialización

Ítem	Acción	Nota
SEC-4	—	Ya cerrado en Lote 2B (era M2).
SEC-1	Ejecutar la auditoría autónoma (7 bloques white+black-box contra staging) del plan existente, con entregable de informe de hallazgos.	Los fixes de la Fase 2 reducen esta auditoría de "descubrimiento" a "confirmación".
SEC-2	B.1: workflow GitHub Actions (smoke API + smoke payments + <code>npm audit</code>) en cada push. B.2: verificación diaria real contra el PJN (requiere release Electron — puede aprovechar la de Lote 2C o una posterior).	B.2 aporta fecha/estado a Diagnóstico del dashboard.

FASE 4 — Cuentas / contratos externos

Ítem	Acción	Prerequisito
B3	Credenciales MP producción al <code>.env</code> del server (por SSH, nunca en repo) + <code>PAYMENT_MODULE_ENABLED=true</code> . Primer cobro real monitoreado de cerca.	Lote 2A en prod y verificado + D4 (lib MP actualizada)
AZ	Azure Trusted Signing: crear cuenta → Certificate Profile (1-3 días hábiles) → configurar electron-builder. Iniciar el trámite temprano (tiene demora externa).	Independiente — se puede tramitar en paralelo desde ya
Facturante	No bloqueante. Activar solo si se contrata (completar <code>FACTURANTE_*</code> + descomentar cron). Mientras, facturación manual.	Independiente

FASE 5 — Diferidos al lanzamiento público

Ítem	Acción	Prerequisito
L1	<code>UPDATE plans SET active=true WHERE name IN ('BASIC', 'PRO', 'ENTERPRISE')</code> — con precios y cobro reales funcionando.	B3 en producción estable
L2	Alimentar el asistente IA con 20-30 tickets reales cerrados.	Volumen de tickets reales

FASE 6 — Módulo Bitácora (propuesta sin aprobar)

Solo si el negocio la aprueba. Diseño completo en `propuesta-bitacora-agenda-2026-07.md` (v6.1).

Sub-fase	Alcance	Release Electron
F1	Backend (3 tablas nuevas + endpoints CRUD, gating por plan) + portal (secciones Bitácora y Mis expedientes). Migraciones aditivas.	No
F2	Botones de captura en los 4 visores + mini-visor de informe individual + paso del tour. Subir el body limit de Nginx/Express a 5MB (config, reversible).	Sí

Bitácora es **puramente aditiva** (tablas nuevas, endpoints nuevos, UI nueva detrás de un flag de plan). Riesgo sobre lo existente: bajo, siempre que el gating quede en off por defecto hasta habilitarlo.

4. Evaluación de riesgo

4.1 Riesgo por fase

Fase	Riesgo sobre producción	Por qué	Mitigación específica
1 — Infra	Bajo (D1, D3, D5) / Alto (D4)	D4 actualiza la librería de MercadoPago con breaking changes	D4 se difiere a justo antes de B3 y se prueba con ciclo de pago sandbox completo en staging
2A — Cobranza	Medio-alto	Toca el corazón del cobro; un error aquí afecta dinero	Staging con ciclo de pago completo; un deploy por bug; MP sandbox; el módulo de pagos real (B3) sigue apagado durante toda la Fase 2
2B — Cuotas	Medio	Cambia enforcement y cons	Probar en staging con usuarios de prueba en trial y pagos; verificar que trials y cuentas ilimitadas no se afecten
2C — Electron	Medio	Requiere release; el auto-updater propaga a todos	Probar <code>npm start</code> + build:dir; fix-forward disponible; GitHub Releases conserva el <code>.exe</code> anterior
2D — U9.3	Bajo	Investigación en staging, no toca prod hasta tener fix	—
3 — Seguridad	Bajo	SEC-1 es lectura; SEC-2 es CI (fuera de runtime prod)	La verificación diaria (B.2) es no bloqueante
4 — Externos	Alto (B3)	Es el primer dinero real	Gate duro: Lote 2A cerrado; primer cobro monitoreado; rollback = apagar <code>PAYMENT_MODULE_ENABLED</code>
5 — Lanzamiento	Bajo	Cambios de datos (activar planes) reversibles	<code>UPDATE ... active=false</code> revierte
6 — Bitácora	Bajo	Puramente aditivo detrás de flag	Gating off por defecto

4.2 Riesgo residual del plan completo

- **El mayor riesgo concentrado es el cobro** (Lote 2A + D4 + B3). Es inherente: hay que tocar el flujo de pagos para poder cobrar bien. La mitigación es que **todo se valida en sandbox con el módulo real apagado**, y B3 solo se enciende cuando el resto está probado.
- **La app Electron tiene el riesgo de propagación** (auto-updater): una release mala llega a todos. Mitigado con fix-forward y pruebas locales previas; conviene agrupar Lote 2C en una sola release bien probada.
- **Riesgo bajo y controlado** en todo lo demás: es aditivo, reversible o fuera del runtime de producción.

4.3 Qué NO cambia (para tranquilidad)

Ninguna fase toca las zonas protegidas: claves RSA (`keys/`), lógica de cifrado/firma (`src/security/` , `utils/scriptEncryption.js`), `machineId` , ni el `manifest.json` de la extensión. Las migraciones son aditivas. Los scripts cifrados solo se re-encryptan si se modifican (no se planea).

5. Backups y recuperación — ¿el `.7z` restablece el proyecto?

Respuesta corta: sí, el `.7z` rutinario de automatización permite reconstruir el proyecto a ese punto en el tiempo (código + base de datos + secretos), que es exactamente lo que hace falta para recuperarse ante un desastre. Con matices importantes que conviene conocer.

5.1 Qué contiene cada .7z (verificado sobre el backup del 2026-07-10)

Archivo dentro del .7z	Qué restaura
procurador_db_backup.sql	Base de datos completa de producción: usuarios, suscripciones, pagos, facturas, tickets, y —clave— la tabla <code>encrypted_scripts</code> con los scripts de automatización cifrados. Es un <code>pg_dump</code> íntegro.
ProcuradorTool_source.7z	Código fuente completo (backend + electron-app + extensión + docs), excluyendo <code>node_modules</code> , <code>dist</code> , <code>.git</code> y <code>.claude</code> .
env_backend.txt	El <code>.env</code> del servidor (todos los secretos: JWT, DB, MP, Anthropic, Brevo).
keys/	Claves RSA (private.pem / public.pem) que firman y verifican los scripts.
certs/	Certificados SSL .

5.2 Qué SÍ se recupera con un .7z

- **Reconstrucción completa del backend en un servidor limpio:** restaurar el `.sql` en PostgreSQL, extraer el código, `npm install` , copiar `.env` + `keys/` + `certs/` , `pm2 start` . El sistema vuelve a funcionar tal cual estaba a la fecha del backup.
- **Reconstrucción de la app Electron desde el fuente:** `npm install` + `npm run build` reproduce el instalador.
- **Los scripts de automatización:** viven en la DB (`encrypted_scripts`), así que el `.sql` los trae. No hace falta re-enciptar.

5.3 Qué NO cubre el .7z (y cómo se cubre por otro lado)

No está en el .7z	Impacto	Red de seguridad alternativa
Historial y tags de git (se excluye <code>.git</code>)	El <code>.7z</code> da una foto del código, no el historial ni los tags de versión	El repo en GitHub es la fuente de verdad del historial + tags de release. Independiente del <code>.7z</code> .
Releases publicados de Electron (<code>.exe</code>)	No se pueden re-descargar del <code>.7z</code>	GitHub Releases conserva cada <code>.exe</code> publicado.
Estado de MercadoPago (preapprovals vivos)	Un restore de DB vieja puede desincronizarse con MP (suscripciones que en MP siguen activas)	Es estado externo; se reconcilia por API de MP + webhooks. No es restaurable por backup por diseño.
Versión en Chrome Web Store	Externo	Se re-sube el ZIP desde el fuente.
Datos creados entre el backup y el incidente	Se pierden los cambios posteriores al último <code>.7z</code>	Backup diario automático 03:00 → DO Spaces (retención 30 días) + <code>ops/backup-now.sh</code> pre-deploy cubren la ventana entre <code>.7z</code> .

5.4 Conclusión sobre recuperación

- **Para código y configuración:** el `.7z` + GitHub (historial/releases) cubren el 100%. Se puede volver a cualquier punto respaldado.
- **Para la base de datos:** el `.7z` da el punto de la sesión; el backup diario a DO Spaces + el pre-deploy on-demand cubren la granularidad fina. Para un rollback de datos preciso se usa `ops/restore-db.sh` (ya probado en simulacro).
- **Recomendación operativa para este plan:** hacer un `.7z` **al inicio y al cierre de cada fase** (no solo por sesión), y un `ops/backup-now.sh prod` **inmediatamente antes de cada deploy del Lote 2A y de B3** (los momentos de mayor riesgo). Con eso, cualquier paso es reversible a su estado previo.

En una frase: sí, con los `.7z` (complementados por el repo de GitHub y el backup diario a DO Spaces) el proyecto es restaurable a los puntos respaldados; lo único no restaurable por backup es el estado de servicios externos (MercadoPago, Chrome Store), que por naturaleza se reconcilia por sus propias APIs.

6. Checklist maestro de ejecución (por deploy)

Para cada cambio que llega a producción:

- [] Tag `pre-<lte>;<lte>;<fecha>` creado.

- [] Cambio probado en staging (backend) o `npm start` + `build:dir` (Electron).
 - [] `ops/backup-now.sh prod` ejecutado (o `.7z` de inicio de fase).
 - [] Migración DB (si aplica) aditiva y con nota de rollback.
 - [] Deploy de UN cambio lógico.
 - [] `smoke-test-pjn.js` + verificación del endpoint/flujo tocado.
 - [] Confirmación de que trials, cuentas ilimitadas y flujos existentes siguen OK.
 - [] Commit + push (historial en GitHub).
 - [] Si es release Electron: bump versión + tag `electron-vX.Y.Z` + `npm run release` + actualizar versión visible en portal.
-

7. Resumen de orden recomendado

1. **Fase 1** (D5 → D1 → D3; D4 se difiere a antes de B3).
2. **Fase 2 Lote 2A** (C1 → C2+A1+M4 → A4 → A2+A3) — cierra el prerequisite de B3.
3. **Fase 2 Lote 2B** (M2/SEC-4, M1, M3, B2, B4, B5, B8).
4. **Fase 2 Lote 2C** (release Electron: A5+A6, M5, M6, M8, M10, B1, B6).
5. **Fase 2 Lote 2D** (investigar U9.3).
6. **Fase 3** (SEC-1 auditoría, SEC-2 CI + verificación diaria).
7. **Fase 1 D4** + **Fase 4 B3** (juntos: lib MP fresca + activar cobro real). AZ y Facturante en paralelo desde antes.
8. **Fase 5** (L1 tras B3 estable, L2 con volumen de tickets).
9. **Fase 6** (Bitácora, si se aprueba: F1 → F2).